



ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



Báo cáo môn học
KỸ THUẬT LẬP TRÌNH

Chủ đề:
Quản lý chi tiêu cá nhân

Giảng viên hướng dẫn: **TS. Vũ Thành Nam**

Nhóm số 69: Hứa Ngọc Khánh **202419070**
 Phạm Thị Ninh **202419086**

Mã lớp: **169313**

Hà Nội, 6/2026

1	Lời mở đầu	1
2	Phân tích bài toán	2
2.1	Giới thiệu bài toán	2
2.1.1	Bối cảnh và vấn đề	2
2.1.2	Mục tiêu hệ thống	2
2.2	Phân tích bài toán	3
3	Thiết kế chương trình	5
3.1	Tổng thể chức năng	5
3.2	Luồng nghiệp vụ hệ thống	6
3.3	Đối tượng	8
3.3.1	User	8
3.3.2	Category	8
3.3.3	Transaction	8
3.3.4	Budget	9
3.3.5	Report	9
3.4	Mô hình thực thể liên kết (ERD)	10
3.5	Mô tả về cấu trúc chương trình	10
3.6	Xây dựng các module	11
3.6.1	Module Data	11
3.6.2	Module Models	12
3.6.3	Module Repositories	13
3.6.4	Module Services	14
3.6.5	Module UI	15
3.6.6	Hàm main	16
4	Gỡ lỗi và kiểm thử	17
4.1	Xác định và gỡ lỗi	17
4.2	Kiểm thử	17
4.2.1	Mục đích kiểm thử	18
4.2.2	Phương pháp kiểm thử	18
5	Các kĩ thuật sử dụng	19
5.1	Kỹ thuật đặt tên và chú thích	19
5.2	Kỹ thuật sử dụng hàm	20
5.3	Kỹ thuật quản lý và xử lý dữ liệu	21
5.4	Kỹ thuật trình bày mã	21
6	Phụ lục hàm main	23
6.1	Mã nguồn hàm main	23
6.2	Phương thức khởi tạo	23
6.3	Các phương thức xử lý và điều hướng giao diện	23

6.3.1	Xóa giao diện hiện tại	23
6.3.2	Xử lý đăng nhập	23
6.3.3	Xây dựng giao diện chính	24
6.3.4	Điều hướng chức năng	25
6.3.5	Xử lý đăng xuất	26
7	Phân công công việc	27
8	Phụ lục mã nguồn	28

1

Lời mở đầu

Lời đầu tiên, chúng em xin gửi lời cảm ơn sâu sắc đến thầy Vũ Thành Nam. Sự giảng dạy, hỗ trợ và tạo điều kiện của thầy đã giúp chúng em hoàn thành tốt nhất báo cáo này. Chúc thầy có thật nhiều sức khỏe để tiếp tục công tác giảng dạy và nghiên cứu khoa học trong thời gian tới!

Trong cuộc sống hiện đại, việc quản lý tài chính cá nhân ngày càng trở nên quan trọng. Tuy nhiên, nhiều người vẫn gặp khó khăn trong việc theo dõi các khoản thu nhập, chi tiêu cũng như kiểm soát ngân sách của bản thân. Việc ghi chép thủ công bằng sổ sách hoặc bảng tính thường mất nhiều thời gian, dễ sai sót và khó tổng hợp khi khối lượng dữ liệu ngày càng lớn.

Xuất phát từ nhu cầu đó, nhóm đã xây dựng hệ thống **Quản lý chi tiêu cá nhân** nhằm hỗ trợ người dùng lưu trữ, theo dõi và thống kê các khoản thu chi một cách thuận tiện. Hệ thống cho phép quản lý giao dịch, thiết lập ngân sách theo từng danh mục, cảnh báo khi vượt hạn mức chi tiêu và tạo các báo cáo trực quan giúp người dùng đánh giá tình hình tài chính của mình.

Thông qua đề tài này, nhóm không chỉ vận dụng các kiến thức về lập trình hướng đối tượng, cấu trúc dữ liệu và giải thuật đã học mà còn có cơ hội tiếp cận quy trình thiết kế và xây dựng một hệ thống phần mềm hoàn chỉnh, từ phân tích yêu cầu, thiết kế kiến trúc đến triển khai và kiểm thử hệ thống.

2

Phân tích bài toán

2.1 Giới thiệu bài toán

2.1.1 Bối cảnh và vấn đề

- Nhu cầu quản lý tài chính cá nhân ngày càng trở nên cần thiết trong cuộc sống hiện đại.
- Các khoản thu và chi phát sinh liên tục theo thời gian, tạo ra khối lượng lớn dữ liệu giao dịch.
- Người dùng thường gặp khó khăn trong việc:
 - Theo dõi các khoản thu chi.
 - Kiểm soát ngân sách theo từng danh mục.
 - Đánh giá tình hình tài chính cá nhân.
 - Tổng hợp số liệu để lập báo cáo.
- Các phương pháp quản lý truyền thống như sổ tay hoặc Excel có nhiều hạn chế:
 - Dễ thất lạc hoặc sai sót dữ liệu.
 - Khó thống kê khi số lượng giao dịch lớn.
 - Hạn chế khả năng kiểm soát ngân sách.
- Khi số lượng giao dịch tăng lên, hệ thống cần có khả năng:
 - Lưu trữ dữ liệu hiệu quả.
 - Tìm kiếm và thống kê nhanh.
 - Hỗ trợ tạo báo cáo theo nhiều khoảng thời gian khác nhau.

2.1.2 Mục tiêu hệ thống

- Xây dựng hệ thống quản lý chi tiêu cá nhân có giao diện trực quan và dễ sử dụng.
- Hỗ trợ người dùng:
 - Đăng ký và đăng nhập tài khoản.
 - Quản lý thông tin cá nhân.
- Thiết lập ngân sách cho từng danh mục chi tiêu theo tháng.
- Cho phép ghi nhận và quản lý các giao dịch:
 - Giao dịch thu.
 - Giao dịch chi.
 - Lưu trữ lịch sử giao dịch.

- Tự động cập nhật ngân sách sau mỗi giao dịch phát sinh.
- Cảnh báo khi mức chi tiêu vượt quá ngân sách đã thiết lập.
- Hỗ trợ trực quan thống kê và báo cáo theo từng khoảng thời gian tùy chọn
- Trực quan hóa dữ liệu bằng biểu đồ và cung cấp nhận xét hỗ trợ người dùng đánh giá tình hình tài chính.
- Vận dụng các cấu trúc dữ liệu và thuật toán đã học để tổ chức, quản lý và xử lý dữ liệu trong hệ thống.

2.2 Phân tích bài toán

Bài toán đặt ra là xây dựng một hệ thống quản lý chi tiêu cá nhân bằng ngôn ngữ Python, sử dụng thư viện Tkinter để xây dựng giao diện người dùng và Matplotlib để trực quan hóa dữ liệu. Hệ thống cho phép người dùng quản lý các khoản thu, chi, thiết lập ngân sách và theo dõi tình hình tài chính cá nhân thông qua các báo cáo thống kê trực quan.

Dữ liệu của hệ thống phải được lưu trữ bền vững để đảm bảo khả năng khôi phục sau khi chương trình được khởi động lại. Đồng thời, hệ thống cần hỗ trợ các chức năng tìm kiếm, thống kê, cảnh báo vượt ngân sách và tạo báo cáo theo nhiều khoảng thời gian khác nhau.

Input

- **Dữ liệu nhập từ giao diện người dùng**
 - Thông tin đăng ký và đăng nhập tài khoản.
 - Thông tin giao dịch thu, chi.
 - Thông tin ngân sách theo từng danh mục.
 - Thông tin danh mục chi tiêu và thu nhập.
- **Dữ liệu đọc từ file JSON**
 - users.json
 - transactions.json
 - budgets.json
 - categories.json
 - reports.json
- Các dữ liệu đọc từ file được chuyển đổi thành các đối tượng và cấu trúc dữ liệu tương ứng trong bộ nhớ để phục vụ xử lý.

Output

- **Lịch sử giao dịch**

- Hiển thị danh sách các khoản thu và chi.
- Hỗ trợ tra cứu và xóa giao dịch.
- **Cảnh báo ngân sách**
 - Phát hiện các danh mục vượt ngân sách.
 - Hiển thị thông báo cảnh báo cho người dùng.
- **Báo cáo thống kê**
 - Tổng hợp thu nhập và chi tiêu.
 - Tính toán số dư hiện tại.
 - Thống kê chi tiêu theo từng danh mục.
 - Tính tỷ lệ chi tiêu của từng danh mục.
- **Biểu đồ trực quan**
 - Biểu đồ tỷ lệ chi tiêu theo danh mục.
 - Biểu đồ được xây dựng bằng thư viện Matplotlib.
- **Kết quả phân tích tài chính**
 - Tổng thu.
 - Tổng chi.
 - Số dư còn lại.
 - Các danh mục vượt ngân sách.

3

Thiết kế chương trình

3.1 Tổng thể chức năng

• Quản lý Giao dịch (Transaction)

- **Ghi chép giao dịch mới:** Hệ thống cho phép người dùng thêm khoản thu/chi thực tế.
 - * Số tiền phải là số thực dương lớn hơn 0.
 - * Danh sách danh mục hiển thị trong Dropdown (Combobox) tự động thay đổi dựa trên Loại giao dịch (Thu/Chi) mà người dùng thao tác chọn trước đó.
 - * Ngay sau khi thêm thành công một giao dịch thuộc loại Chi tiêu, hệ thống tự động kiểm tra ngân sách của danh mục tương ứng.
- **Xóa giao dịch:** Cho phép xóa hoàn toàn khoản chi tiêu/thu nhập đã nhập sai thông qua ID. Mọi thao tác xóa giao dịch chỉ đều làm cập nhật lại tổng chi thực tế của danh mục, ảnh hưởng trực tiếp đến trạng thái cảnh báo của Ngân sách.
- **Xem lịch sử giao dịch:** Lịch sử được hiển thị dưới dạng bảng. Cho phép người dùng lọc nhanh danh sách theo khoảng thời gian (Từ ngày - Đến ngày) và theo Loại giao dịch (Tất cả / Thu nhập / Chi tiêu).

• Quản lý Ngân sách (Budget)

- **Đặt hạn mức ngân sách:** Cho phép thiết lập số tiền tối đa được phép chi tiêu cho một danh mục trong một tháng cụ thể.
 - * Mỗi danh mục chỉ chỉ được phép có duy nhất 01 ngân sách trong cùng một tháng. Nếu đã tồn tại, hệ thống tự động chuyển sang chế độ ghi đè cập nhật hạn mức mới.
 - * Hạn mức ngân sách phải là số thực dương.
- **Kiểm tra và Cảnh báo vượt mức:** Hệ thống liên tục so sánh tổng số tiền chi thực tế của danh mục với hạn mức đã đặt. Nếu số tiền chi vượt qua hạn mức ngân sách, hệ thống kích hoạt cảnh báo trực quan trên giao diện (đổi màu thanh tiến trình sang màu đỏ và xuất hiện popup cảnh báo rủi ro) để báo động cho người dùng.

• Báo cáo và Thống kê (Report)

- **Tổng hợp số liệu kỳ:** Tự động quét và tính toán dữ liệu trong khoảng thời gian tùy chọn (ngày, tuần, tháng, năm).
 - * Tính tổng thu (`totalIncome`) và tổng chi (`totalExpense`).
 - * Tính toán Số dư hiện tại ($\text{Balance} = \text{totalIncome} - \text{totalExpense}$) và hiển thị tổng quan.
 - * Tính tổng số tiền đã chi chi tiết theo từng danh mục và quy ra tỉ lệ phần trăm tương ứng.

- **Xuất báo cáo đồ họa:** Trích xuất dữ liệu từ `categorySummaries` và sử dụng thư viện `Matplotlib` để kết xuất biểu đồ đồ họa. Dùng biểu đồ tròn (Pie chart) để thể hiện rõ nét tỷ trọng chi tiêu giữa các nhóm danh mục.
- **Quản lý lưu trữ dữ liệu**
 - **Đọc dữ liệu từ file:** Khi khởi động ứng dụng, kiến trúc Repository tự động tải toàn bộ nội dung từ các file `users.json`, `categories.json`, `budgets.json`, `reports.json` và `transactions.json` vào bộ nhớ.
 - **Ghi dữ liệu xuống file:** Sau mỗi thao tác (Thêm / Sửa / Xóa) làm thay đổi dữ liệu của thực thể, hệ thống tự động gọi phương thức `_save_all()` của lớp `BaseRepository` để ghi đè trạng thái hiện tại của bộ nhớ xuống file vật lý.
 - * Đảm bảo tính bền vững của dữ liệu nếu chương trình bị đóng hoặc mất điện đột ngột.
 - * Cấu trúc JSON đảm bảo lưu trữ chuẩn xác mảng các đối tượng (objects) với đầy đủ key-value tương ứng theo thuộc tính lớp.

3.2 Luồng nghiệp vụ hệ thống

Hệ thống quản lý chi tiêu cá nhân được xây dựng nhằm hỗ trợ người dùng theo dõi các khoản thu, chi, quản lý ngân sách và tạo báo cáo tài chính. Luồng nghiệp vụ của hệ thống được mô tả như sau:

- **Đăng ký tài khoản**
 - Người dùng nhập địa chỉ email và mật khẩu.
 - Hệ thống kiểm tra tính hợp lệ của email và mật khẩu.
 - Nếu thông tin hợp lệ, tài khoản được lưu vào cơ sở dữ liệu.
- **Đăng nhập hệ thống**
 - Người dùng nhập email và mật khẩu đã đăng ký.
 - Hệ thống xác thực thông tin đăng nhập.
 - Nếu thông tin chính xác, người dùng được phép truy cập các chức năng của hệ thống.
- **Thiết lập ngân sách**
 - Người dùng tạo ngân sách theo từng tháng.
 - Người dùng thiết lập ngân sách cho từng danh mục chi tiêu.
 - Ngân sách được sử dụng để theo dõi và kiểm soát chi tiêu.
- **Nhập giao dịch**
 - Chọn ngày thực hiện giao dịch.
 - Chọn loại giao dịch (thu hoặc chi).

- Nhập ghi chú (không bắt buộc).
- Nhập số tiền giao dịch.
- Chọn danh mục giao dịch.
- Xác nhận lưu giao dịch.
- **Cập nhật ngân sách và lưu trữ dữ liệu**
 - Tự động cập nhật số tiền đã sử dụng của danh mục tương ứng.
 - Trừ ngân sách đối với các giao dịch chi tiêu (nếu danh mục đã được thiết lập ngân sách).
 - Lưu thông tin giao dịch vào cơ sở dữ liệu.
 - Cập nhật lịch sử giao dịch để phục vụ tra cứu và thống kê.
- **Kiểm tra ngân sách và cảnh báo**
 - Kiểm tra mức sử dụng ngân sách của từng danh mục.
 - Xác định các danh mục có chi tiêu vượt ngân sách.
 - Hiển thị cảnh báo để người dùng điều chỉnh kế hoạch chi tiêu.
- **Tạo báo cáo tài chính**
 - Cho phép lựa chọn khoảng thời gian báo cáo:
 - * Theo ngày.
 - * Theo tuần.
 - * Theo tháng.
 - * Theo quý.
 - * Theo năm.
 - * Khoảng thời gian tùy chọn.
 - Tổng hợp các khoản thu, chi và tính toán số dư.
 - Phân loại giao dịch theo danh mục.
 - Tính tỷ lệ chi tiêu của từng danh mục.
 - Xác định các danh mục vượt ngân sách.
- **Trực quan hóa dữ liệu và nhận xét**
 - Tạo biểu đồ thể hiện tỷ lệ chi tiêu theo từng danh mục.
 - Hiển thị các thông tin thống kê trực quan.
 - Đưa ra nhận xét bằng văn bản về tình hình tài chính của người dùng.
- **Đăng xuất**
 - Kết thúc phiên làm việc hiện tại.
 - Xóa thông tin đăng nhập khỏi bộ nhớ tạm của ứng dụng.
 - Quay trở lại màn hình đăng nhập ban đầu.

3.3 Đối tượng

Để đảm bảo hệ thống Quản lý chi tiêu cá nhân hoạt động ổn định và đáp ứng đầy đủ các luồng nghiệp vụ cốt lõi, cơ sở dữ liệu được thiết kế và mô hình hóa thành 5 lớp đối tượng trung tâm: User (Người dùng), Category (Danh mục), Transaction (Giao dịch), Budget (Ngân sách) và Report (Báo cáo). Các đối tượng này có mối liên kết chặt chẽ với nhau theo mô hình quan hệ. Cấu trúc dữ liệu chi tiết của từng lớp đối tượng được trình bày lần lượt trong các bảng dưới đây:

3.3.1 User

Tên Thuộc Tính	Kiểu Dữ Liệu	Mô Tả Chức Năng
id	int	Mã định danh người dùng (Khóa chính)
name	string	Họ và tên hiển thị của người dùng
email	string	Địa chỉ email dùng để đăng ký/đăng nhập vào hệ thống
password	string	Mật khẩu đăng nhập hệ thống
currency	string	Đơn vị tiền tệ hiển thị mặc định (VD: VND, USD)
created_at	datetime	Thời gian tài khoản được tạo lập vào hệ thống

Bảng 1: Cấu trúc dữ liệu của User

3.3.2 Category

Tên Thuộc Tính	Kiểu Dữ Liệu	Mô Tả Chức Năng
id	int	Mã định danh danh mục (Khóa chính)
name	string	Tên gọi của danh mục (VD: Ăn uống, Tiền nhà, Tiền lương)
type	string	Phân loại danh mục thuộc nhóm Thu nhập hay Chi tiêu

Bảng 2: Cấu trúc dữ liệu của Category

3.3.3 Transaction

Tên Thuộc Tính	Kiểu Dữ Liệu	Mô Tả Chức Năng
id	int	Mã định danh giao dịch (Khóa chính)
amount	decimal	Số tiền thu vào hoặc chi ra của giao dịch
type	string	Phân loại luồng tiền (Thu hoặc Chi)
note	string	Ghi chú, diễn giải thêm về mục đích của giao dịch (Tùy chọn)
date	date	Ngày tháng năm thực hiện giao dịch thực tế
created_at	datetime	Dấu thời gian hệ thống ghi nhận bản ghi này

Bảng 3: Cấu trúc dữ liệu của Transaction

3.3.4 Budget

Tên Thuộc Tính	Kiểu Dữ Liệu	Mô Tả Chức Năng
id	int	Mã định danh ngân sách (Khóa chính)
limit_amount	decimal	Hạn mức số tiền tối đa được phép chi tiêu cho danh mục
month	string	Tháng/Kỳ áp dụng hạn mức ngân sách này
created_at	datetime	Thời điểm bản ghi ngân sách được thiết lập

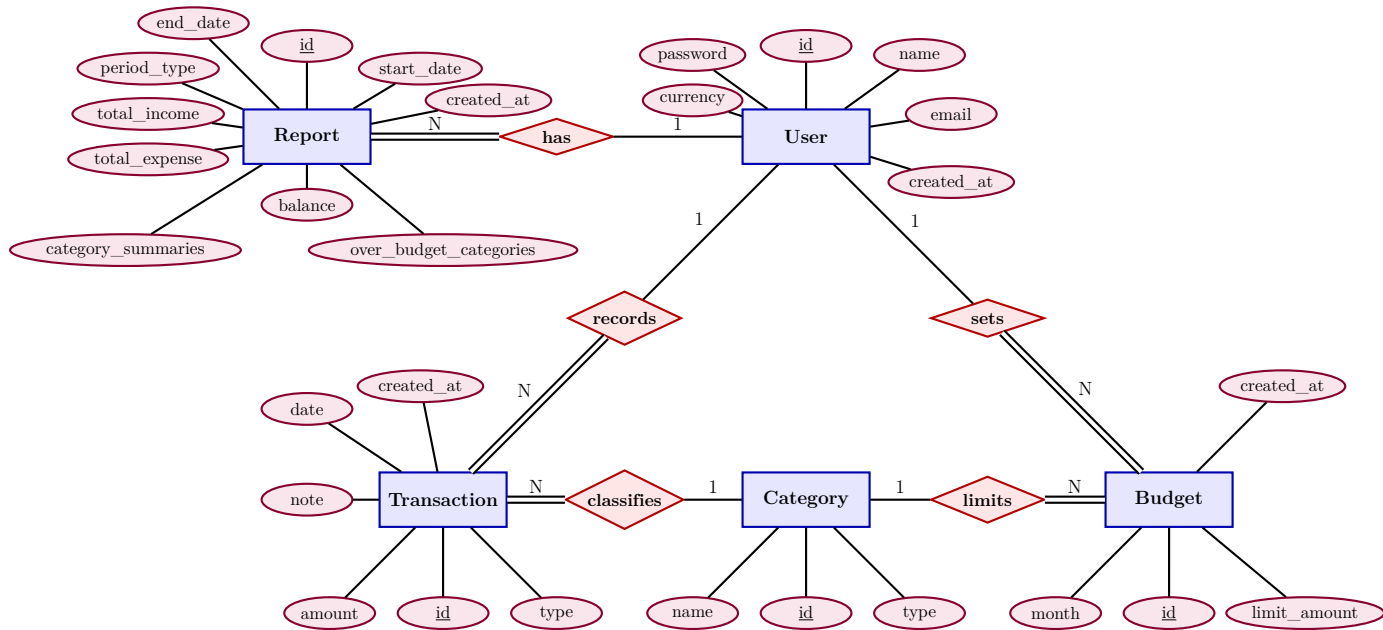
Bảng 4: Cấu trúc dữ liệu của Budget

3.3.5 Report

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của báo cáo
start_date	string	Ngày bắt đầu kỳ báo cáo (YYYY-MM-DD)
end_date	string	Ngày kết thúc kỳ báo cáo (YYYY-MM-DD)
period_type	string	Loại kỳ (day, week, month, quarter, year, custom)
total_income	float	Tổng thu nhập trong kỳ
total_expense	float	Tổng chi tiêu trong kỳ
balance	float	Số dư (total_income - total_expense)
created_at	string	Thời gian tạo báo cáo (YYYY-MM-DD HH:MM:SS)
over_budget_categories	LinkedList	Danh sách các danh mục vượt hạn mức
category_summaries	LinkedList	Chi tiết thống kê danh mục (tên, chi, hạn mức, %)

Bảng 5: Cấu trúc dữ liệu của Report

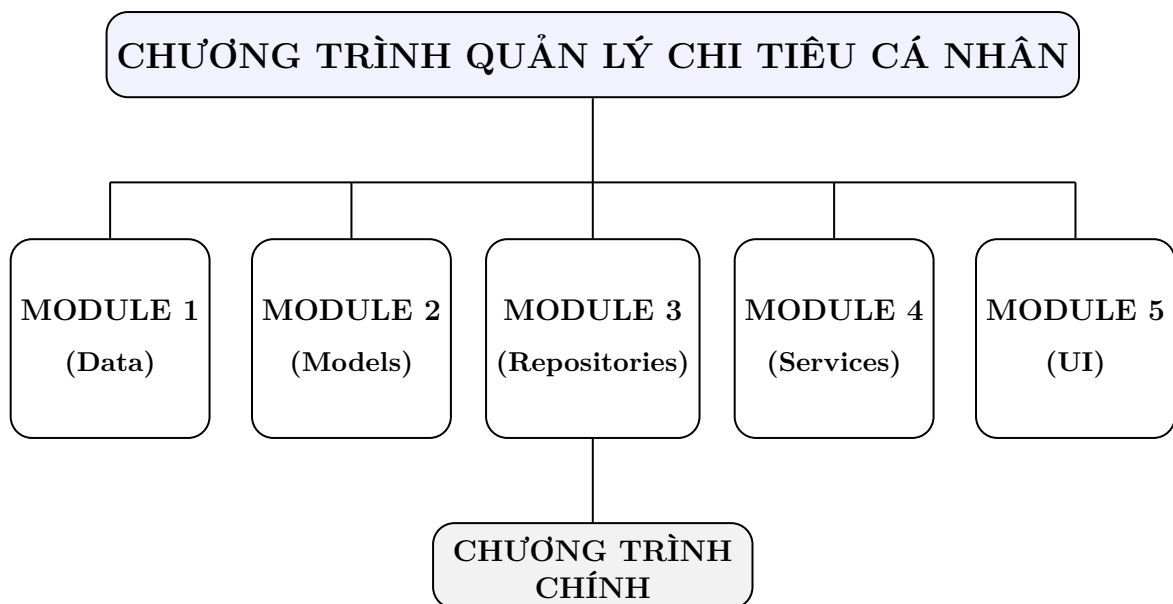
3.4 Mô hình thực thể liên kết (ERD)



Hình 3.1: Mô hình thực thể liên kết (ERD)

3.5 Mô tả về cấu trúc chương trình

Dưới đây là sơ đồ mô tả về cấu trúc chương trình:



Hình 3.2: Sơ đồ phân rã cấu trúc Module theo Kiến trúc phân tầng (Layered Architecture)

Bên cạnh đó, dưới đây là sơ đồ cấu trúc thư mục chứa dự án:

Dự án Quản lý chi tiêu cá nhân/

```
|--- __pycache__           # Thư mục chứa các file .pyc do Python sinh ra
|--- data                  # Module 1: Data
|   |--- budgets.json
|   |--- categories.json
|   |--- reports.json
|   |--- transactions.json
|   |--- users.json
|--- models                # Module 2: Models
|   |--- linked_list.py
|   |--- models.py
|--- repositories          # Module 3: Repositories
|   |--- base_repository.py
|   |--- repositories.py
|--- services              # Module 4: Services
|   |--- budget_service.py
|   |--- report_service.py
|   |--- transaction_service.py
|   |--- user_service.py
|--- ui                    # Module 5: UI
|--- main.py               # Chương trình chính
```

Hình 3.3: Cấu trúc thư mục mã nguồn và phân bổ theo kiến trúc 5 Module

3.6 Xây dựng các module

3.6.1 Module Data

Module Data chịu trách nhiệm lưu trữ dữ liệu lâu dài của hệ thống.

- Lưu trữ dữ liệu trong các tệp JSON:
 - users.json
 - categories.json
 - transactions.json
 - budgets.json
 - reports.json
- Quản lý tập trung toàn bộ dữ liệu trong thư mục data/.

- Chuẩn hóa dữ liệu theo định dạng JSON nhằm:
 - Dễ đọc và kiểm tra.
 - Dễ chuyển đổi giữa đối tượng và dữ liệu lưu trữ.
 - Thuận tiện cho việc gỡ lỗi và bảo trì.
- Đảm bảo dữ liệu được lưu trữ lâu dài và phục hồi khi khởi động lại chương trình.
- Đồng bộ trạng thái giữa hệ thống và dữ liệu vật lý trên ổ đĩa.

3.6.2 Module Models

Trong Module 2, hệ thống tiến hành định nghĩa các đối tượng lõi theo nguyên lý lập trình hướng đối tượng (OOP). Các cấu trúc này được tổ chức trong thư mục `models/`, bao gồm 5 lớp thực thể chính: `User`, `Category`, `Transaction`, `Budget`, `Report` được khai báo trong file `models.py`, cùng với cấu trúc dữ liệu tự cài đặt `Node` và `LinkedList` nằm trong file `linked_list.py`.

Về cấu trúc dữ liệu tự cài đặt:

- Lớp `Node` được thiết kế theo hướng tổng quát hóa, chỉ bao gồm hai thành phần chính:
 - `data`: lưu trữ dữ liệu của phần tử.
 - `next`: con trỏ tham chiếu đến phần tử kế tiếp trong danh sách.
- Lớp `LinkedList` quản lý toàn bộ cấu trúc danh sách liên kết thông qua:
 - `head`: con trỏ đến phần tử đầu tiên của danh sách.
 - `size`: biến theo dõi số lượng phần tử hiện có trong danh sách.
- Ngoài các thao tác cơ bản như `append`, `get`, `remove`, hệ thống còn cài đặt thêm các phương thức nâng cao:
 - `find`: tìm kiếm phần tử theo điều kiện (predicate function).
 - `filter`: lọc các phần tử thỏa mãn điều kiện.
 - `to_list`: chuyển đổi toàn bộ cấu trúc `LinkedList` sang danh sách Python (list) để phục vụ cho việc tuần tự hóa dữ liệu khi ghi file JSON.

Về các lớp thực thể: Tất cả các lớp thực thể trong hệ thống đều được xây dựng theo chuẩn OOP và tích hợp các phương thức nền tảng sau:

- `__init__`: phương thức khởi tạo đối tượng, hỗ trợ gán giá trị mặc định cho các thuộc tính như thời gian tạo và số tiền.
- `__repr__`: phương thức biểu diễn kỹ thuật của đối tượng, phục vụ cho mục đích debug và kiểm tra dữ liệu.

- `to_dict`: chuyển đổi đối tượng Python thành dictionary để chuẩn hóa dữ liệu trước khi lưu trữ.
- `from_dict`: khôi phục đối tượng từ dictionary, hỗ trợ quá trình đọc dữ liệu từ file JSON và tái tạo object.

3.6.3 Module Repositories

Module Repositories đóng vai trò tầng truy cập dữ liệu giữa Data và Services.

- **Base Repository**
 - Quản lý thao tác CRUD thông qua lớp `BaseRepository`.
 - Tự động kiểm tra và khởi tạo file JSON rỗng nếu chưa tồn tại.
 - Nạp dữ liệu từ file JSON vào bộ nhớ dưới dạng `LinkedList`.
 - Đồng bộ dữ liệu từ bộ nhớ xuống tệp JSON sau mỗi thao tác thay đổi.
 - Tự động sinh ID cho các bản ghi mới.
- **Repositories:**
 - **UserRepository:**
 - * Quản lý dữ liệu người dùng từ file `data/users.json`.
 - * Hỗ trợ tìm kiếm người dùng theo email thông qua phương thức `get_by_email()`.
 - * Chuẩn hóa email trước khi so sánh bằng cách loại bỏ khoảng trắng thừa và chuyển về chữ thường.
 - **CategoryRepository:**
 - * Quản lý dữ liệu danh mục từ file `data/categories.json`.
 - * Hỗ trợ truy vấn danh mục theo loại: thu nhập, chi tiêu hoặc cả hai.
 - * Tự động khởi tạo danh mục mặc định khi hệ thống chạy lần đầu nếu dữ liệu rỗng.
 - **TransactionRepository:**
 - * Quản lý dữ liệu giao dịch từ file `data/transactions.json`.
 - * Hỗ trợ truy vấn theo người dùng, loại giao dịch, danh mục và khoảng thời gian.
 - * Cung cấp phương thức `get_expense_by_category_in_month()` để tính tổng chi tiêu theo danh mục trong tháng.
 - * Phục vụ chức năng kiểm tra và đối chiếu ngân sách.
 - **BudgetRepository:**
 - * Quản lý dữ liệu ngân sách từ file `data/budgets.json`.
 - * Hỗ trợ truy vấn theo người dùng, tháng và danh mục cụ thể.
 - * Dùng để đối chiếu chi tiêu thực tế với hạn mức ngân sách.
 - **ReportRepository:**

- * Quản lý dữ liệu báo cáo từ file `data/reports.json`.
- * Hỗ trợ truy vấn theo người dùng và loại kỳ báo cáo (ngày, tuần, tháng, quý, năm).
- * Hỗ trợ tra cứu báo cáo theo khoảng thời gian đã xác định.

3.6.4 Module Services

Module Services đóng vai trò tầng xử lý logic nghiệp vụ, nằm giữa Repositories và UI.

- Các dịch vụ cốt lõi (Services):
 - **UserService:**
 - * Quản lý logic đăng ký, đăng nhập và xác thực người dùng.
 - * Giao tiếp với **UserRepository** để đối chiếu thông tin tài khoản và mật khẩu đầu vào.
 - * Kiểm tra tính hợp lệ của thông tin đăng ký (chống trùng lặp email, tính hợp lệ của định dạng email, độ an toàn).
 - * Quản lý trạng thái phiên đăng nhập (session) của người dùng hiện tại.
 - **TransactionService:**
 - * Cung cấp các phương thức CRUD mở rộng: thêm mới, chỉnh sửa, xóa bản ghi giao dịch.
 - * Kiểm tra tính hợp lệ của dữ liệu đầu vào (ví dụ: số tiền phải > 0 , đúng định dạng ngày tháng) trước khi chuyển xuống tầng Repository.
 - * Lọc và tính toán tổng thu nhập, tổng chi tiêu và số dư hiện tại dựa trên danh sách lịch sử giao dịch.
 - * Xử lý phân nhóm giao dịch theo danh mục để phục vụ cho các chức năng thống kê nâng cao.
 - **BudgetService:**
 - * Chịu trách nhiệm kiểm soát và theo dõi tiến độ sử dụng ngân sách của người dùng.
 - * Truy xuất và đối chiếu tổng chi tiêu thực tế (từ **TransactionRepository**) với hạn mức đã đặt (từ **BudgetRepository**).
 - * Tính toán chi tiết các chỉ số: ngân sách khả dụng (remaining budget), số tiền đã chi, số tiền vượt hạn mức.
 - * Đánh giá tỷ lệ chi tiêu và tạo dữ liệu cảnh báo trạng thái (over_budget) khi tổng chi vượt quá hạn mức ngân sách đã thiết lập.
 - **ReportService:**
 - * Tổng hợp dữ liệu thô từ các dịch vụ khác để kết xuất các bảng thống kê tài chính định kỳ.
 - * Chuẩn bị và định dạng bộ dữ liệu (dataset) với cấu trúc tối ưu để truyền sang module UI phục vụ vẽ biểu đồ.
 - * Hỗ trợ trích xuất báo cáo tổng quan đa chiều (theo thời gian, theo tỷ trọng các danh mục chi tiêu).

3.6.5 Module UI

Module UI đóng vai trò hiển thị giao diện đồ họa và tiếp nhận các tương tác đầu vào từ người dùng.

- Thành phần giao diện (Frames):
 - `theme.py`:
 - * Lưu trữ và quản lý các hằng số về thiết kế trực quan, quy định bộ mã màu chuẩn.
 - * Thiết lập cấu hình **Font** chữ thống nhất cho từng cấp độ (Tiêu đề, Văn bản thường, Nút bấm).
 - * Đảm bảo tính nhất quán về mặt giao diện (UI consistency) trên toàn bộ các màn hình của hệ thống.
 - `login_frame.py`:
 - * Cung cấp giao diện chuyển đổi linh hoạt giữa hai chức năng: Đăng nhập (Login) và Đăng ký (Register).
 - * Xây dựng các biểu mẫu với **Entry** nhập liệu, đặc biệt sử dụng chế độ ẩn ký tự (`show="•"`) để bảo mật trường mật khẩu.
 - * Thu thập văn bản nhập từ bàn phím, gọi module Services xác thực và hiển thị lỗi (nếu có).
 - * Tích hợp widget `messagebox` để hiển thị các popup cảnh báo, lỗi xác thực hoặc thông báo đăng ký thành công trực quan.
 - `dashboard_frame.py`:
 - * Thiết kế bố cục chia vùng: tích hợp thanh điều hướng (Sidebar) giúp người dùng chuyển đổi nhanh giữa các tính năng.
 - * Xây dựng khu vực thẻ tóm tắt (Summary Cards) trực quan hóa số liệu tài chính realtime bằng **Label** (Số dư, Tổng thu, Tổng chi).
 - * Tích hợp widget **Treeview** cấu hình đa cột để hiển thị danh sách lịch sử giao dịch.
 - * Bố trí thêm **Scrollbar** (thanh cuộn) liên kết với **Treeview** để tối ưu hóa việc xem dữ liệu khi danh sách quá dài.
 - `transaction_frame.py`:
 - * Xây dựng biểu mẫu (Form) chi tiết cho phép người dùng thêm các giao dịch thu/chi mới.
 - * Ứng dụng **Combobox** để người dùng lựa chọn Loại giao dịch và Danh mục từ danh sách có sẵn, giảm thiểu lỗi nhập liệu thủ công.
 - * Xử lý xác thực dữ liệu đầu vào trực tiếp trên form (client-side validation) trước khi gửi yêu cầu tới **TransactionService**.
 - * Cung cấp cơ chế làm sạch form (clear form) tự động sau khi giao dịch được thêm thành công.
 - `budget_report_frame.py`:

- * Xây dựng giao diện xem báo cáo với các bộ lọc (Filter) bằng Combobox cho phép chọn tháng/năm cần thống kê.
- * Tích hợp thư viện đồ họa để nhúng dữ liệu vào Canvas, hiển thị các biểu đồ thể hiện tỷ trọng chi tiêu.
- * Thiết kế các thanh tiến trình (Progress Bar) trực quan hóa mức độ tiêu hao của hạn mức ngân sách và tự động cảnh báo (đổi màu đỏ) khi vượt quá hạn mức.
- * Cập nhật lại biểu đồ và dữ liệu tự động mỗi khi người dùng thay đổi tiêu chí lọc.

3.6.6 Hàm main

Module Main đóng vai trò điều phối trung tâm, áp dụng mô hình kiến trúc MVC để kết nối các module rời rạc thành một hệ thống hoàn chỉnh.

- Khởi tạo hệ thống (App):
 - Khởi tạo khung giao diện gốc của ứng dụng và thiết lập các cấu hình màn hình ban đầu.
 - Các Repository liên kết trực tiếp sẽ do các lớp Service chủ động khởi tạo tại hàm `__init__` một cách độc lập, `main.py` chỉ quản lý luồng ứng dụng và phiên đăng nhập của người dùng.
- Quản lý luồng màn hình:
 - Cung cấp phương thức `_navigate()` để điều hướng linh hoạt giữa các màn hình chức năng.
 - Cơ chế hoạt động dựa trên việc hủy bỏ (destroy) toàn bộ các nội dung (widget) của Frame hiện tại, sau đó khởi tạo lại (recreate) class Frame của màn hình mới (như `DashboardFrame`, `BudgetFrame`) và dùng `pack()` để hiển thị lên vùng nhìn chính.
- Vòng lặp sự kiện (Mainloop):
 - Chứa khối `if __name__ == "__main__":` tiêu chuẩn nhằm đảm bảo mã nguồn chỉ thực thi khi file script được khởi chạy trực tiếp.
 - Kích hoạt trạng thái vòng lặp bằng lệnh `mainloop()` để duy trì cửa sổ hoạt động và liên tục lắng nghe tương tác từ chuột/bàn phím của người dùng.

4

Gỡ lỗi và kiểm thử

4.1 Xác định và gỡ lỗi

Tìm kiếm và xác định lỗi là quá trình xác định xem chương trình có lỗi không bằng cách:

- Quan sát kết quả sai
- So sánh với kết quả đầu ra mong đợi
- Kiểm thử các tình huống bất thường

Việc tìm kiếm lỗi cú pháp trong quá trình viết chương trình là tương đối dễ dàng vì trong Python sẽ hiển thị lỗi lên màn hình ngay. Không chỉ vậy IDE cũng thông báo lỗi cú pháp ở file nào trong thư mục được mở và số lượng lỗi trong file đó ngay trên màn hình. Đối với lỗi logic thì khó phát hiện hơn vì phải thực hiện debug hoặc kiểm thử thì mới phát hiện ra. Khi này, những điều sau hỗ trợ rất nhiều trong việc hạn chế và phát hiện lỗi:

- Phân tách chương trình thành các module rõ ràng để dễ dàng truy dấu lỗi
- Viết mã rõ ràng, dễ theo dõi
- Sử dụng công cụ debugger của IDE để theo dõi luồng chạy
- Luôn làm việc với phiên bản mã nguồn mới nhất bằng cách sử dụng GitHub, đảm bảo không debug lại phiên bản cũ
- Sử dụng các cấu trúc try-except để định vị các loại lỗi tiềm ẩn
- Phát hiện và debug ngay trong khi kiểm thử

Sau khi đã xác định được lỗi thông qua quá trình kiểm thử, thông báo lỗi hay hành vi bất thường của chương trình, việc cần làm là gỡ lỗi. Các bước gỡ lỗi thường trải qua các bước sau:

- **Bước 1: Tìm nguyên nhân** : Xem xét logic code, kiểm tra biến, kiểm tra điều kiện rẽ nhánh...
- **Bước 2: Sửa lỗi** : Chỉnh sửa phần code gây ra lỗi
- **Bước 3: Kiểm tra lại** : Chạy lại chương trình để đảm bảo lỗi đã được khắc phục và không gây ra lỗi mới

4.2 Kiểm thử

Kiểm thử là quá trình kiểm tra hay đánh giá một hệ thống hay một thành phần hệ thống một cách thủ công hay tự động để kiểm chứng rằng nó thỏa mãn những yêu cầu đặc thù hoặc để xác định sự khác biệt giữa kết quả mong đợi và kết quả thực tế.

4.2.1 Mục đích kiểm thử

Quá trình kiểm thử được tiến hành nhằm đảm bảo phần mềm vận hành đúng theo các yêu cầu và kỳ vọng đã đề ra từ đầu. Mục tiêu chính của kiểm thử là phát hiện sớm và xử lý kịp thời các lỗi về logic và chức năng, từ đó nâng cao độ tin cậy và tính ổn định của phần mềm trong quá trình xử lý dữ liệu. Đồng thời, kiểm thử cũng nhằm đánh giá hiệu năng tổng thể của phần mềm, bao gồm khả năng xử lý nhanh chóng và hiệu quả các thao tác, cũng như mức độ thân thiện và dễ sử dụng của giao diện người dùng. Qua đó, đảm bảo hệ thống không chỉ đáp ứng yêu cầu kỹ thuật mà còn mang lại trải nghiệm tốt cho người sử dụng cuối cùng.

4.2.2 Phương pháp kiểm thử

Những test case được trình bày bên dưới được áp dụng phương pháp kiểm thử hộp đen (Black-box testing) và kiểm thử hộp trắng (White-box testing) ở các khía cạnh khác nhau. Ngoài ra, chiến lược hồi quy và chiến lược kiểm thử hệ thống cũng được chúng em áp dụng trong quá trình kiểm thử. Cụ thể, sau mỗi lần sửa lỗi và cải tiến chương trình, em phải kiểm thử lại bằng cách chạy các test case của các chức năng cũ để đảm bảo việc thêm mới không gây ra xung đột giữa các luồng thực hiện chức năng, điều có thể làm hỏng chương trình. Trong quá trình đó, toàn bộ chương trình được kiểm tra như một hệ thống hoàn chỉnh.

5

Các kỹ thuật sử dụng

Hệ thống được cài đặt bằng Python theo hình mẫu lập trình hướng đối tượng, sử dụng cấu trúc dữ liệu tự cài đặt LinkedList thay vì các cấu trúc có sẵn, lưu trữ dữ liệu dưới dạng file JSON, và xây dựng giao diện đồ hoạ bằng Tkinter kết hợp Matplotlib.

5.1 Kỹ thuật đặt tên và chú thích

Đặt tên là kỹ thuật nền tảng trong lập trình có cấu trúc, quyết định trực tiếp đến khả năng đọc hiểu và bảo trì của mã nguồn. Theo nguyên tắc kỹ thuật lập trình kinh điển Clean Code của Robert C. Martin, một tên gọi tốt cần thoả mãn:

- Có nghĩa và thể hiện đúng mục đích
- Nhất quán về quy ước trên toàn dự án
- Phân biệt rõ vai trò qua quy ước
- Độ dài tương xứng với phạm vi sử dụng

Chú thích được dùng để bổ sung ngữ cảnh, lý do thiết kế, hoặc cảnh báo tác dụng phụ mà tên hàm không thể truyền tải hết.

Quy ước đặt tên áp dụng trong hệ thống được mô tả như trong bảng dưới

Thành phần	Quy ước	Ví dụ trong mã nguồn
Tên lớp (Class)	PascalCase, danh từ	TransactionService, BudgetRepository, LinkedList
Tên hàm/method	snake_case, động từ	add_transaction(), get_month_status(), _validate_date()
Tên biến	snake_case, danh từ	total_income, budget_limit, category_id
Hằng số	UPPER_SNAKE_CASE	DEFAULT_EXPENSE_CATEGORIES, ALLOWED_TLDS
Phương thức private	tiền tố dấu gạch dưới _	_validate_month(), _check_budget(), _next_id()
Tên file module	snake_case, số ít	transaction_service.py, budget_repository (trong file)

Ngoài ra, hệ thống sử dụng hai loại chú thích với mục đích khác nhau: docstring ở đầu các module, class, hàm và inline comment cho các đoạn chương trình. Ví dụ, trong hàm `validate_and_normalize_amount` sử dụng cả 2 loại chú thích này

```
1 def _validate_and_normalize_amount(self, amount):
2     """
3     Xác thực và chuẩn hóa số tiền.
4     - Chuyển đổi thành float
5     - Kiểm tra > 0
6     - Làm tròn 2 chữ số thập phân
7     Trả về (is_valid: bool, amount_normalized: float,
8         error_message: str)
9     """
10    if not amount:
11        return False, None, "Số tiền không được để trống."
12
13    try:
14        # Chuyển đổi thành float (đã được chuẩn hóa từ UI)
15        amount_float = float(amount)
16    except (ValueError, TypeError):
17        return False, None, "Số tiền không hợp lệ. Vui lòng nhập số."
18
19    # Kiểm tra > 0
20    if amount_float <= 0:
21        return False, None, "Số tiền phải lớn hơn 0."
22
23    # Làm tròn 2 chữ số thập phân
24    amount_normalized = round(amount_float, 2)
25
26    return True, amount_normalized, ""
```

5.2 Kỹ thuật sử dụng hàm

Hàm, thủ tục hay chương trình con là đơn vị phân rã cơ bản của lập trình có cấu trúc, dựa trên nguyên lý chia để trị: một bài toán lớn được tách thành các bài toán con độc lập, mỗi bài toán con được giải quyết bởi một hàm có trách nhiệm duy nhất. Ví dụ, trong mã nguồn của hệ thống, chức năng Thêm giao dịch không được viết thành một hàm khổng lồ duy nhất, mà được phân rã thành chuỗi các hàm con, mỗi hàm đảm nhiệm đúng một bước:

```
1 # transaction_service.py
2 def add_transaction(self, user_id, category_id, amount, type,
3     note="", date=None):
4     # Bước 1: Xác thực và chuẩn hóa số tiền
5     is_valid, amount_normalized, error_msg = self.
6         _validate_and_normalize_amount(amount)
7     if not is_valid:
8         raise ValueError(error_msg)
9
10    if type not in ("income", "expense"):
```

```

9         raise ValueError("Loại giao dịch phải là 'income' hoặc '
           expense'.")
10
11     # Bước 2: Xác thực ngày
12     date_to_use = date or datetime.now().strftime("%Y-%m-%d")
13     is_valid, error_msg = self._validate_date(date_to_use)
14     if not is_valid:
15         raise ValueError(error_msg)
16
17     # Bước 3: Tạo và lưu đối tượng
18     tx = Transaction(id=0, user_id=user_id, category_id=
           category_id,
19                     amount=amount_normalized, type=type, note=
           note, date=date_to_use)
20     saved_tx = self.tx_repo.add(tx)
21
22     # Bước 4: Kiểm tra ngân sách (chỉ với khoản chi)
23     alert = None
24     if type == "expense":
25         alert = self._check_budget(user_id, category_id, saved_tx
           .date)
26     return saved_tx, alert

```

5.3 Kỹ thuật quản lý và xử lý dữ liệu

Hệ thống sử dụng Danh sách liên kết đơn tự cài đặt trong tầng `models` làm cấu trúc lưu trữ dữ liệu chính trong bộ nhớ RAM thay vì mảng hay list có sẵn của Python. Lớp này cung cấp các thao tác cơ bản như `append`, `get`, `remove`, `find` và `filter`. Toàn bộ dữ liệu load từ JSON lên đều được chuyển thành đối tượng và lưu trong `LinkedList`. Các lớp `Repository` sẽ sử dụng hàm `filter()` để duyệt tuần tự và trích xuất dữ liệu thỏa mãn điều kiện thành một danh sách mới.

Ngoài ra, dữ liệu của hệ thống được lưu trữ bền vững cục bộ dưới định dạng văn bản JSON. Cụ thể, trong thư mục `data`, dữ liệu được lưu trữ, thực hiện ghi và lấy thông tin ở các files `budgets.json`, `categories.json`, `reports.json`, `transactions.json`, `user.json`. Hai quá trình chính trong việc lưu trữ là tuần tự hóa bằng `json.dump()` và giải tuần tự hóa bằng `json.load()`. Mỗi lớp model tự định nghĩa phương thức `to_dict()` và `from_dict()` để thực hiện chuyển đổi này. Đối với mô hình phức tạp như `Report`, các thuộc tính chứa `LinkedList` được gọi hàm `to_list()` chuyển về dạng mảng thường để có thể ghi ra JSON. Hệ thống cũng áp dụng chiến lược lưu trữ `write-through` toàn phần, nghĩa là mỗi khi có một thao tác thay đổi dữ liệu, toàn bộ cấu trúc trong RAM sẽ được ghi đè xuống file JSON.

5.4 Kỹ thuật trình bày mã

Hệ thống sử dụng kiến trúc phân tầng chia chương trình thành các tầng có trách nhiệm riêng biệt, tầng trên chỉ gọi xuống tầng dưới và không có chiều ngược lại.


```
|--- models
|   |--- linked_list.py
|   |--- models.py
|--- repositories
|   |--- base_repository.py
|   |--- repositories.py
|--- services
|   |--- budget_service.py
|   |--- report_service.py
|   |--- transaction_service.py
|   |--- user_service.py
|--- ui
|   |--- budget_report_frame.py
|   |--- dashboard_frame.py
|   |--- login_frame.py
|   |--- theme.py
|   |--- transaction_frame.py
|--- main.py
```

Theo đó, sự phụ thuộc trong toàn bộ mã nguồn chỉ chạy theo một chiều: ui → services → repositories → models. Nhờ đó, mỗi tầng có thể được đọc, kiểm thử hoặc thay đổi mà ít ảnh hưởng tới các tầng còn lại.

6

Phụ lục hàm main

6.1 Mã nguồn hàm main

```
1 if __name__ == "__main__":
2     app = App()
3     app.mainloop()
```

6.2 Phương thức khởi tạo

```
1 class App(tk.Tk):
2     def __init__(self):
3         super().__init__()
4         self.title("Quản Lý Chi Tiêu Cá Nhân")
5         self.geometry("900x680")
6         self.minsize(800, 600)
7         self.configure(bg=COLORS["bg"])
8         apply_theme(self)
9
10        self.current_user = None
11        self._show_login()
```

6.3 Các phương thức xử lý và điều hướng giao diện

6.3.1 Xóa giao diện hiện tại

```
1 def _clear(self):
2     for widget in self.winfo_children():
3         widget.destroy()
```

6.3.2 Xử lý đăng nhập

```
1 def _show_login(self):
2     self._clear()
3     LoginFrame(self, on_login_success=self._on_login).pack(
4         fill="both", expand=True)
5
6 def _on_login(self, user):
7     self.current_user = user
8     self.title(f"Chi Tiêu Cá Nhân - {user.name}")
9     self._show_main()
```

6.3.3 Xây dựng giao diện chính

```

1  def _show_main(self):
2      self._clear()
3
4      # --- Sidebar -----
5      sidebar = tk.Frame(self, bg=COLORS["primary_dark"], width
6                          =180)
7      sidebar.pack(side="left", fill="y")
8      sidebar.pack_propagate(False)
9
10     tk.Label(sidebar, text="Chi Tiêu\nCá Nhân",
11              font=FONTS["subhead"], fg="#FFFFFF",
12              bg=COLORS["primary_dark"],
13              justify="center").pack(pady=(32, 24))
14
15     menu_items = [
16         ("Dashboard", "dashboard"),
17         ("Thêm giao dịch", "add_transaction"),
18         ("Lịch sử", "history"),
19         ("Ngân sách", "budget"),
20         ("Báo cáo", "report"),
21     ]
22     self._nav_btns = {}
23     for label, key in menu_items:
24         btn = tk.Button(
25             sidebar, text=label,
26             font=FONTS["body"],
27             fg="#FFFFFF", bg=COLORS["primary_dark"],
28             activeforeground="#FFFFFF",
29             activebackground=COLORS["primary"],
30             relief="flat", anchor="w",
31             padx=20, pady=10,
32             cursor="hand2",
33             command=lambda k=key: self._navigate(k),
34         )
35         btn.pack(fill="x")
36         self._nav_btns[key] = btn
37
38     # Nút đăng xuất
39     tk.Button(sidebar, text="Đăng xuất",
40              font=FONTS["small"],
41              fg=COLORS["border"], bg=COLORS["primary_dark"],
42              activeforeground="#FFFFFF",
43              activebackground=COLORS["danger"],
44              relief="flat", anchor="w",
45              padx=20, pady=8,
46              cursor="hand2",

```

```

46         command=self._logout).pack(side="bottom", fill=
47         "x",
48         pady=(0, 16))
49
50     # --- Content area -----
51     self.content = tk.Frame(self, bg=COLORS["bg"])
52     self.content.pack(side="right", fill="both", expand=True)
53
54     self._navigate("dashboard")

```

6.3.4 Điều hướng chức năng

```

1     def _navigate(self, screen):
2         # Highlight nút active
3         for key, btn in self._nav_btns.items():
4             btn.config(bg=COLORS["primary"] if key == screen
5                       else COLORS["primary_dark"])
6
7         # Xóa nội dung cũ
8         for w in self.content.winfo_children():
9             w.destroy()
10
11         user = self.current_user
12
13         if screen == "dashboard":
14             frame = DashboardFrame(self.content, user,
15                                   on_navigate=self._navigate)
16         elif screen == "add_transaction":
17             frame = AddTransactionFrame(
18                 self.content, user,
19                 on_save=lambda: self._navigate("dashboard"))
20         elif screen == "history":
21             frame = TransactionListFrame(self.content, user)
22         elif screen == "budget":
23             frame = BudgetFrame(self.content, user)
24         elif screen == "report":
25             frame = ReportFrame(self.content, user)
26         else:
27             return
28
29         frame.pack(fill="both", expand=True)

```

6.3.5 Xử lý đăng xuất

```
1     def _logout(self):  
2         self.current_user = None  
3         self.title("Quản Lý Chi Tiêu Cá Nhân")  
4         self._show_login()
```

7 Phân công công việc

STT	Thành viên	Nội dung công việc	Đánh giá
1	Phạm Thị Ninh	- Phân tích bài toán - Thiết kế chương trình - Phụ lục hàm main	A
2	Hứa Ngọc Khánh	- Gỡ lỗi và kiểm thử - Các kĩ thuật sử dụng	A

Bảng 7.1: Bảng phân công nhiệm vụ chi tiết của các thành viên

8 Phụ lục mã nguồn

Mã nguồn chi tiết của dự án được công khai trên GitHub tại địa chỉ: <https://github.com/khanh-hua-kkk/pmQuanLyChiTieu.git>